PALANTIR FOUNDRY PROJECT ENABLEMENT PROGRAM

Day 1 Hands-On Exercise
Building Your First Ontology Foundation

Phase 1 — Foundation Accelerator — Module 0

Author: Rajesh Pasham

Table of Contents

Overview

In this exercise you will set up a Foundry Developer Tier account (if you don't already have one), create your first two Ontology object types, link them together, and back them with real sample data — entirely within your own personal developer environment.

Estimated time: 40–60 minutes.

What You Will Build

- A Vehicle object type with 8 properties
- A Transponder object type with 6 properties
- A one-to-one link type between them
- Two datasets (vehicles.csv and transponders.csv) backing those object types
- (Stretch goal) A simple Workshop view showing your Vehicle objects in a table

Why generic data: This exercise uses generic sample data (vehicles and transponders) so that it is fully self-contained and runs identically in any personal Foundry Developer Tier account, independent of any specific organization's enrollment. The concepts transfer directly to the real project Ontology you will extend from Sprint 1 onward.

Prerequisites

- A modern web browser (Chrome, Edge, or Firefox)
- The two sample data files provided with this exercise: vehicles.csv and transponders.csv
- No local software installation is required for this exercise

Exercise 1 — Access Your Foundry Developer Tier Account

If you already have access to a Foundry enrollment through this engagement, skip to Exercise 2.

1. Go to the Palantir Developers site (palantir.com/developers) and choose the option to sign up for a free Developer Tier / AIP Developer Tier account.
2. Complete the sign-up form with your work email address and verify your account when prompted.
3. Once your enrollment is provisioned, log in and confirm you land on the Foundry home page.
4. Bookmark your enrollment's home page URL — you will use it as the base for the Ontology Manager URL in the next exercise.

Exercise 2 — Orient Yourself in Ontology Manager

1. From your Foundry home page, add /workspace/ontology to the end of the URL and press Enter.
2. Take note of the top bar (search, create, and branch navigation) and the sidebar (resource navigation).
3. Open the Discover view and review the “Recently viewed object types” and “Favorite object types” sections — both will be empty for a brand-new enrollment.
4. Use the search bar (or Cmd/Ctrl + K) to confirm search is working, even though there is nothing to find yet.

Exercise 3 — Create the Vehicle Object Type

Properties to define:

Property	Data Type	Notes
vehicleId	String	Primary key / title property
vin	String	17-character VIN
make	String	
model	String	
year	Integer	
licensePlate	String	
status	String	Active / Maintenance / Inactive
currentLocation	String	Facility name

1. In Ontology Manager, select New from the top bar and choose Object Type.
2. Set the object type's API name to Vehicle and its display name to Vehicle (singular, PascalCase per our coding standard).
3. Add each property listed in the table above, one at a time, setting its API name and data type to match exactly — property API names must be lowerCamelCase.
4. Set vehicleId as the object type's title property and primary key.
5. Save the object type. It will show as unpublished / draft until it has a backing dataset (Exercise 6).

Exercise 4 — Create the Transponder Object Type

Properties to define:

Property	Data Type	Notes
transponderId	String	Primary key / title property
serialNumber	String	
status	String	Active / In Stock / Faulty
issueDate	Date	ISO format, e.g. 2023-02-14
deviceType	String	e.g. RFID-Gen2, BLE-Beacon, Hybrid
assignedVehicleId	String	Foreign key to Vehicle.vehicleId; blank when unassigned

1. Repeat the process from Exercise 3: New → Object Type, API name and display name Transponder.
2. Add each property from the table above, matching data types exactly.
3. Set transponderId as the title property and primary key.
4. Save the object type.

Exercise 5 — Link Vehicle and Transponder

1. Open the Vehicle object type and navigate to its Links tab.
2. Select New Link Type and choose Transponder as the target object type.
3. Name the link assignedTransponder (Vehicle → Transponder) and installedOnVehicle (Transponder → Vehicle) for the reverse direction.
4. Set the cardinality to one-to-one for this exercise — a vehicle has at most one active transponder today.
5. Map the link using the foreign key: Transponder.assignedVehicleId equals Vehicle.vehicleId.
6. Save the link type.

Why one-to-one for now: Later sprints extend this relationship to one-to-many so that a vehicle's full transponder history can be tracked over time. Modeling it as one-to-one today keeps the first pass simple and is a deliberate teaching choice, not a limitation of Foundry.

Exercise 6 — Upload the Sample Datasets

Two sample data files are provided with this exercise. Their schemas are:

vehicles.csv

```
vehicleId,vin,make,model,year,licensePlate,status,currentLocation
VEH-001,1FTBW3XG7PKA10234,Ford,Transit-350,2022,TRK-1042,Active,Warehouse A
VEH-002,1HTMMAAL5NH123456,International,DuraStar,2021,TRK-2087,Active,Distribution
Center 2
... (12 rows total)
```

transponders.csv

```
transponderId,serialNumber,status,issueDate,deviceType,assignedVehicleId
TRP-1001,SN-880231,Active,2023-02-14,RFID-Gen2,VEH-001
TRP-1002,SN-880232,Active,2023-02-14,RFID-Gen2,VEH-002
... (14 rows total, 3 unassigned)
```

1. From the Foundry home page, select New → Dataset → Upload File.
2. Upload vehicles.csv, name the dataset Vehicles Raw, and place it in a project folder for this engagement.
3. Review the inferred schema: confirm year is inferred as an integer/long and every other column as a string. Correct any column the importer infers incorrectly.
4. Repeat for transponders.csv, naming the dataset Transponders Raw. Confirm issueDate is inferred as a date or string — if it is inferred as a string, that is fine for this exercise.
5. Build (run) both datasets so their data is available for the next step.

Exercise 7 — Back Your Object Types With Real Data

1. Return to the Vehicle object type in Ontology Manager and open its Datasources / Mapping configuration.
2. Select the Vehicles Raw dataset as the backing datasource.
3. Map each dataset column to the matching property one-to-one (vehicleId → vehicleId, vin → vin, and so on).
4. Confirm vehicleId is mapped as the primary key, then publish the object type.
5. Repeat the same mapping process for Transponder using the Transponders Raw dataset.
6. Publish the Transponder object type.

Exercise 8 — Verify Your Objects

1. In Ontology Manager, open the Vehicle object type and select its Objects / Explore tab.
2. Confirm all 12 vehicle records appear, with correct values for make, model, and status.
3. Open the Transponder object type and confirm all 14 transponder records appear.
4. Open one Vehicle object (e.g. VEH-001) and confirm its linked Transponder (TRP-1001) appears through the assignedTransponder link.
5. Open a Transponder with no assignedVehicleId (e.g. TRP-1009) and confirm it correctly shows no linked Vehicle.

Troubleshooting: If a linked object doesn't appear, double check the foreign key mapping in Exercise 5 and that both datasets were rebuilt after any schema correction.

Exercise 9 (Stretch Goal) — A First Look at Workshop

If time allows, take a first look at how Workshop consumes the Ontology you just built:

1. From the Foundry home page, select New → Workshop.
2. Add an Object Table widget to the canvas and bind it to the Vehicle object type.
3. Add a Filter widget bound to the status property and confirm filtering to Active narrows the table.
4. Save your Workshop module with a name such as Vehicle Overview (Day 1 Practice).

Appendix A — Full Dataset Reference

vehicles.csv (12 rows)

```
vehicleId,vin,make,model,year,licensePlate,status,currentLocation
VEH-001,1FTBW3XG7PKA10234,Ford,Transit-350,2022,TRK-1042,Active,Warehouse A
VEH-002,1HTMMAAL5NH123456,International,DuraStar,2021,TRK-2087,Active,Distribution
Center 2
VEH-003,4V4NC9EH7DN123789,Volvo,VNL 760,2023,TRK-3311,Active,Warehouse A
VEH-004,3AKJHHDR5JSJA1234,Freightliner,Cascadia,2020,TRK-4456,Maintenance,Service
Bay 1
VEH-005,1FUJGLDR8CLBP1234,Kenworth,T680,2022,TRK-5521,Active,Distribution Center 2
VEH-006,1XPBDP9X1ND123456,Peterbilt,579,2019,TRK-6098,Inactive,Warehouse B
VEH-007,JALC4W16007001234,Isuzu,NPR-HD,2023,TRK-7213,Active,Warehouse A
VEH-008,JHHFC1H50GK123456,Hino,268A,2021,TRK-8034,Active,Distribution Center 1
VEH-009,1FTBW3XG2NKA56789,Ford,Transit-250,2024,TRK-9145,Active,Warehouse B
VEH-010,4V4NC9EHXEN654321,Volvo,VNL 860,2020,TRK-1056,Maintenance,Service Bay 2
VEH-011,1HTMMAAL2PH987654,International,LT Series,2024,TRK-1187,Active,Distribution
Center 1
VEH-012,3AKJHHDR1LSJB5678,Freightliner,Cascadia,2022,TRK-1298,Active,Warehouse A
```

transponders.csv (14 rows)

```
transponderId,serialNumber,status,issueDate,deviceType,assignedVehicleId
TRP-1001,SN-880231,Active,2023-02-14,RFID-Gen2,VEH-001
TRP-1002,SN-880232,Active,2023-02-14,RFID-Gen2,VEH-002
TRP-1003,SN-880233,Active,2023-03-01,RFID-Gen3,VEH-003
TRP-1004,SN-880234,Faulty,2022-11-20,RFID-Gen2,VEH-004
TRP-1005,SN-880235,Active,2023-05-09,RFID-Gen3,VEH-005
TRP-1006,SN-880236,Active,2021-09-17,RFID-Gen2,VEH-006
TRP-1007,SN-880237,Active,2023-07-22,BLE-Beacon,VEH-007
TRP-1008,SN-880238,Active,2023-07-22,BLE-Beacon,VEH-008
TRP-1009,SN-880239,In Stock,2024-01-10,RFID-Gen3,
TRP-1010,SN-880240,In Stock,2024-01-10,RFID-Gen3,
TRP-1011,SN-880241,Active,2024-02-18,Hybrid,VEH-009
TRP-1012,SN-880242,Active,2020-12-05,RFID-Gen2,VEH-010
TRP-1013,SN-880243,Active,2024-04-02,Hybrid,VEH-011
TRP-1014,SN-880244,In Stock,2024-04-02,Hybrid,
```

Appendix B (Advanced / Optional) — Code-Based Transform

If you prefer defining your pipeline in code rather than uploading raw files directly, this optional Python transform (for a Code Repository) cleans the raw Vehicles dataset before it reaches the Ontology — for example, standardizing status values and dropping any rows missing a vehicleId.

```
from transforms.api import transform_df, Input, Output
from pyspark.sql import functions as F

@transform_df(
    Output("/Project/datasets/vehicles_clean"),
    raw=Input("/Project/datasets/vehicles_raw"),
)
def compute(raw):
    df = raw.dropna(subset=["vehicleId"])
    df = df.withColumn("status", F.initcap(F.trim(F.col("status"))))
    df = df.withColumn("year", F.col("year").cast("int"))
    return df.dropDuplicates(["vehicleId"])
```

A preview of what Day 2 looks like: once your Ontology is published, this is roughly how a Python OSDK application will read your Vehicle objects.

```
from foundry_sdk import FoundryClient
from foundry_sdk.auth import UserTokenAuth

auth = UserTokenAuth(hostname="<YOUR_ENROLLMENT>.palantirfoundry.com")
client = FoundryClient(auth=auth, hostname="<YOUR_ENROLLMENT>.palantirfoundry.com")

# Iterate over all Active vehicles
for vehicle in client.ontology.objects.Vehicle.where(
    client.ontology.objects.Vehicle.status == "Active"
).iterate():
    print(vehicle.vehicleId, vehicle.make, vehicle.model, vehicle.currentLocation)
```

Definition of Done — Day 1

You have completed today's hands-on exercise when:

- The Vehicle object type is published with all 8 properties and backed by real data
- The Transponder object type is published with all 6 properties and backed by real data
- The assignedTransponder link correctly connects at least 10 Vehicle/Transponder pairs
- You can browse both object types and their linked objects in Ontology Manager
- (Stretch) A simple Workshop table view displays your Vehicle objects

Looking ahead: Keep your Developer Tier enrollment — you will extend this same Ontology in tomorrow's OSDK Architecture module.